Laboratório 09

6LoWPAN: Cabeçalhos, Compressão e Fragmentação

Tecnologias de Comunicação para a Internet das Coisas | Aula 09

Objetivo e enquadramento

A Aula 09 apresenta o 6LoWPAN como camada de adaptação que torna possível transportar IPv6 em tramas IEEE 802.15.4 de 127 octetos. Este laboratório reproduz, com um pequeno programa em Python, os cálculos e os mecanismos da aula: o orçamento de uma trama, a compressão de cabeçalhos, a fragmentação e a construção do identificador de interface.

O trabalho não requer rádio, hardware nem máquina virtual: corre em qualquer computador com Python 3. Destina-se a ser realizado durante a aula, ao ritmo de cada aluno, e **não constitui trabalho a entregar**.

Nota

Os valores são calculados e não medidos num rádio real. Os bytes da compressão são, contudo, interpretados pela biblioteca `scapy` como cabeçalhos 6LoWPAN, o que permite verificar a sua validade.

Resultados de aprendizagem

No final deste laboratório, o aluno deve ser capaz de:

- Calcular quantos octetos de carga útil restam numa trama 802.15.4, com e sem segurança.
- Identificar os campos do cabeçalho IPHC e do NHC de UDP e relacioná-los com a compressão máxima de 6 octetos.
- Comparar o custo da compressão com endereços de ligação local e com endereços globais.
- Estimar o número de tramas necessárias para uma mensagem e o efeito das perdas sobre a

fragmentação.

- Construir o identificador de interface a partir do identificador da PAN e do endereço curto.

Requisitos

- Computador com Windows, macOS ou Linux e Python 3.9 ou superior.
- Ligação à Internet para instalar o `scapy`.
- Duração estimada: entre 40 e 60 minutos.

Como utilizar este guião

Cada passo apresenta, por esta ordem, o objetivo, o enquadramento, o fundamento teórico, os comandos a executar e o resultado esperado. As caixas têm sempre o mesmo significado:

Comando a executar

```
Comando a executar no terminal.
```

Resultado esperado (os valores podem variar)

```
Resultado que deve ser apresentado.
```

Analisar

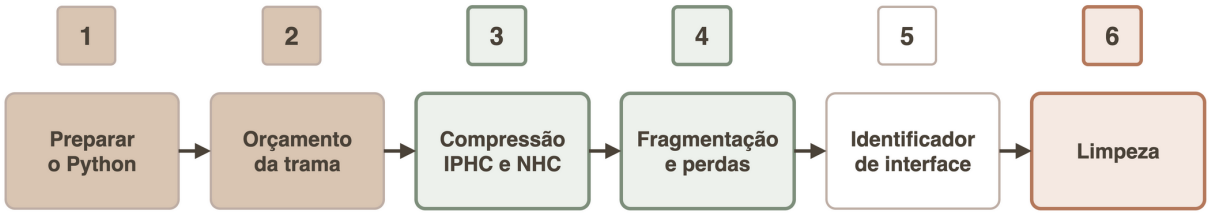
Momento de pausa para interpretar o resultado obtido, isoladamente ou em discussão com os colegas.

Erro possível e resolução

Erro que pode ocorrer e respetiva resolução.

Os resultados mostrados foram obtidos com o programa deste guião e devem coincidir com os do aluno.

Percurso



Tudo corre num único programa Python; os números são os mesmos dos diapositivos

Como ler a figura

Os seis passos leem-se da esquerda para a direita. Os dois primeiros, a bege, preparam o programa e calculam o orçamento da trama. Os passos 3 e 4, a verde, são a parte central: a compressão dos cabeçalhos e a fragmentação. O passo 5, a branco, constrói o identificador de interface, e o passo 6 encerra o trabalho.

Passo 1 — Preparação do Python

Objetivo. Criar um ambiente isolado, instalar o `scapy` e guardar o programa do laboratório.

Fundamento teórico

Um ambiente virtual do Python é uma pasta que contém uma cópia isolada do interpretador e das bibliotecas. O `scapy` é uma biblioteca de manipulação de pacotes que conhece o formato dos cabeçalhos 6LoWPAN, e por isso serve para construir pacotes IPv6 e para validar os bytes que o programa produz.

1.1 Ambiente e biblioteca

Comando a executar

```
python3 -m venv ~/tciot-lab09
~/tciot-lab09/bin/pip install scapy
```

Em Windows, utilizar:

Comando a executar

```
python -m venv tciot-lab09
tciot-lab09\Scripts\pip install scapy
```

1.2 O programa

Criar o ficheiro `sixlowpan_lab.py` com o conteúdo seguinte. O ficheiro está também disponível na pasta `codigo` do material da aula.

Conteúdo do ficheiro `sixlowpan_lab.py`

```
# /// script
# requires-python = ">=3.9"
# dependencies = ["scapy"]
# ///
"""6LoWPAN lab (Lecture 09): header budget, IPHC/NHC compression, fragmentation,
    interface identifier.

Run:  uv run sixlowpan_lab.py [payload_bytes]
```

```
"""
import logging
import sys
from math import ceil

logging.getLogger("scapy.runtime").setLevel(logging.CRITICAL)
from scapy.layers.inet6 import IPv6, UDP
from scapy.layers.sixlowpan import LoWPAN_IPHC

FRAME = 127                 # IEEE 802.15.4 maximum frame size
MAC_OPEN, MAC_SEC = 25, 46  # MAC header without / with AES-CCM-128


def budget(ipv6_udp_header, mac):
    """Octets left for the application payload in one frame."""
    return FRAME - mac - ipv6_udp_header


def compress(link_local=True, ports=(0xF0B1, 0xF0B2)):
    """Minimal IPHC + NHC(UDP) encoder, returns the compressed header bytes.

    link_local=True : both addresses elided (derived from the link-layer addresses
    )
    link_local=False: both addresses carried inline (128 bits each)
    """
    hlim = 0b10                                     # hop limit 64 elided
    b0 = (0b011 << 5) | (0b11 << 3) | (1 << 2) | hlim   # 011 | TF=11 elided | NH
    =1 NHC follows | HLIM
    sam = dam = 0b11 if link_local else 0b00
    b1 = (0 << 7) | (0 << 6) | (sam << 4) | (0 << 3) | (0 << 2) | dam
    out = bytes([b0, b1])
    if not link_local:
        out += bytes(32)                            # placeholder for the two inline
     addresses
    nhc = 0b11110_0_11                              # UDP NHC, checksum inline, both
     ports 4 bits
    out += bytes([nhc, ((ports[0] & 0xF) << 4) | (ports[1] & 0xF)]) + b"\x00\x00"
    return out


def fragments(compressed_header, payload, mac):
    """Number of 802.15.4 frames needed (simplified: FRAG1 = 4 octets, FRAGN = 5
    octets of overhead)."""
    data = len(compressed_header) + payload
    if data <= FRAME - mac:
        return 1
    first = ((FRAME - mac - 4) // 8) * 8
    rest = ((FRAME - mac - 5) // 8) * 8
```

```
    return 1 + ceil((data - first) / rest)


def short_to_iid(pan, short):
    """PAN:0000:short -> interface identifier (FFFE in the middle, U/L bit set to
    0 = local)."""
    b = pan.to_bytes(2, "big") + b"\x00\x00" + short.to_bytes(2, "big")
    iid = bytearray(b[:3] + b"\xff\xfe" + b[3:])
    iid[0] &= 0xFD
    return ":".join(f"{iid[i]:02x}{iid[i+1]:02x}" for i in range(0, 8, 2))

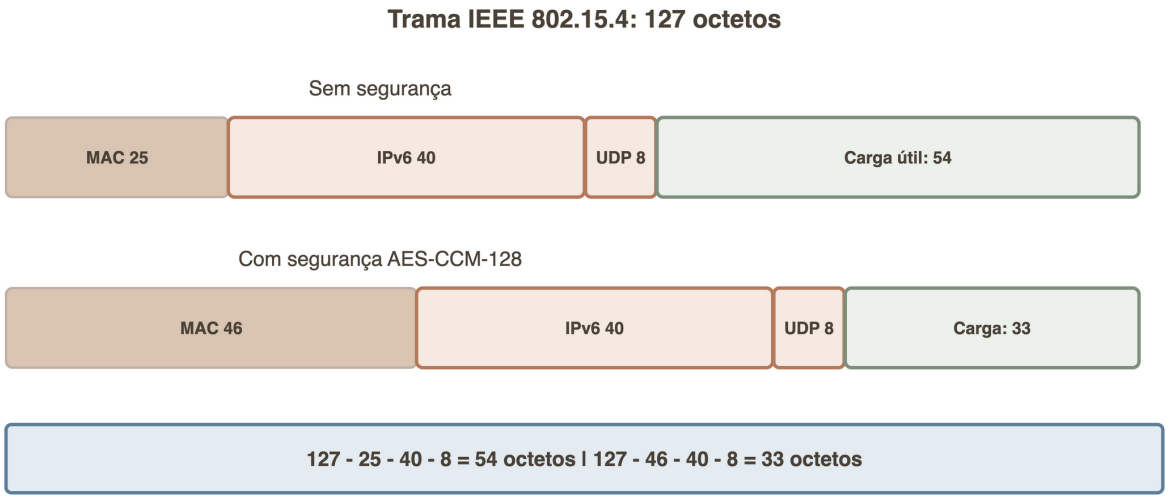

if __name__ == "__main__":
    app = int(sys.argv[1]) if len(sys.argv) > 1 else 20
    pkt = IPv6(src="fe80::1", dst="fe80::2") / UDP(sport=5683, dport=5683) / (b"x"
     * app)
    hdr = len(pkt) - app
    print(f"IPv6 + UDP header: {hdr} octets; application payload: {app} octets")
    for name, mac in (("without security", MAC_OPEN), ("with AES-CCM-128", MAC_SEC
    )):
        print(f"  frame budget {name}: {budget(hdr, mac)} octets left")
    for ll in (True, False):
        c = compress(link_local=ll)
        kind = "link-local" if ll else "global addresses inline"
        print(f"IPHC+NHC ({kind}): {len(c)} octets  ->  {budget(len(c), MAC_OPEN)}
     octets left")
    c = compress(True)
    print("bytes:", c.hex(" "))
    print("scapy sees IPHC:", LoWPAN_IPHC(c).summary())
    for size in (20, 60, 150, 400):
        n_u = fragments(bytes(hdr), size, MAC_OPEN)
        n_c = fragments(compress(True), size, MAC_OPEN)
        print(f"payload {size:3d} octets: {n_u} frame(s) uncompressed, {n_c}
    compressed")
    for p in (0.01, 0.1):
        print(f"loss {p:.0%} per frame: success with 1 frame {(1-p):.2f}, with 5
    frames {(1-p)**5:.2f}")
    print("interface id for PAN 0xABCD, short 0x0001:", short_to_iid(0xABCD, 0
    x0001))
```

Erro possível e resolução

ModuleNotFoundError: No module named 'scapy': o programa foi executado com o Python do sistema e não com o do ambiente virtual. Usar `~/tciot-lab09/bin/python sixlowpan_lab.py`.

Passo 2 — O orçamento da trama

Objetivo. Calcular quantos octetos restam para a aplicação numa trama 802.15.4.



Como ler a figura

A figura é a mesma do diapositivo. As barras têm largura proporcional ao número de octetos: o cabeçalho MAC, o IPv6 e o UDP a bege e a vermelho, e a carga útil a verde. A barra de cima não tem segurança e a de baixo usa AES-CCM-128, que alarga o cabeçalho MAC de 25 para 46 octetos.

Fundamento teórico

A trama 802.15.4 tem no máximo 127 octetos. O cabeçalho IPv6 ocupa 40 octetos e o UDP ocupa 8, sem qualquer compressão. Subtraindo a estes valores o cabeçalho MAC, obtêm-se 54 octetos de carga útil sem segurança e 33 com segurança AES-CCM-128. A função `budget` do programa faz exatamente esta conta.

A analisar neste passo: a proporção entre cabeçalho e dados. Mais de metade da trama é cabeçalho.

Comando a executar

```
~/tciot-lab09/bin/python sixlowpan_lab.py 20
```

Resultado esperado (os valores podem variar)

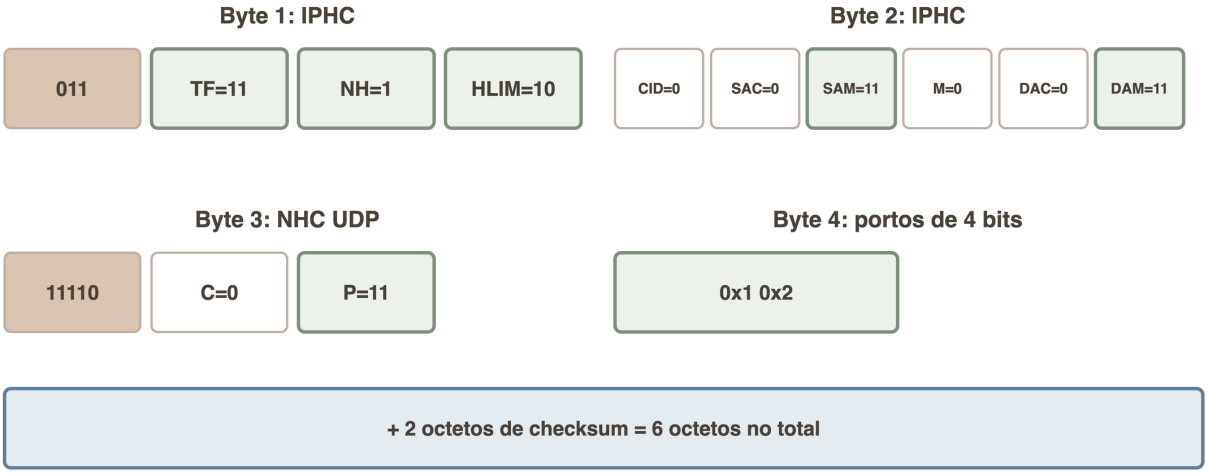
```
IPv6 + UDP header: 48 octets; application payload: 20 octets
  frame budget without security: 54 octets left
  frame budget with AES-CCM-128: 33 octets left
```

Analisar

Altere o argumento para 100 e repita. O orçamento muda? O que aconteceria à mensagem de 100 octetos de carga útil?

Passo 3 — Compressão com IPHC e NHC

Objetivo. Observar os bytes do cabeçalho comprimido e comparar o custo com endereços de ligação local e com endereços globais.



Como ler a figura

Os dois primeiros octetos formam o cabeçalho IPHC. O primeiro indica que o campo de classe de tráfego e etiqueta de fluxo é omitido, que o cabeçalho seguinte usa NHC e que o limite de saltos 64 é omitido. O segundo indica que os dois endereços são omitidos, por serem derivados dos endereços de camada 2. O terceiro octeto é o NHC de UDP, e o quarto transporta os portos de 4 bits. A soma, com os dois octetos de *checksum*, é de 6 octetos.

Fundamento teórico

A compressão sem estado omite tudo o que o recetor consegue deduzir: a versão, o comprimento, o limite de saltos habitual e, em ligação local, os endereços. Os portos UDP no intervalo 61616 a 61631 (`0xF0B0` a `0xF0BF`) cabem em 4 bits cada. Quando os endereços são globais, não podem ser deduzidos a partir da camada 2 e têm de ser transportados, o que faz subir o cabeçalho.

A analisar neste passo: que campos desapareceram e porque o recetor os consegue reconstituir.

Comando a executar

```
~/tciot-lab09/bin/python sixlowpan_lab.py 20 | sed -n 4,7p
```

Resultado esperado (os valores podem variar)

```
IPHC+NHC (link-local): 6 octets  ->  96 octets left
IPHC+NHC (global addresses inline): 38 octets  ->  64 octets left
bytes: 7e 33 f3 12 00 00
scapy sees IPHC: LoWPAN_IPHC / LoWPAN_NHC / IPv6 / UDP :::61617 > :::61618
```

Analisar

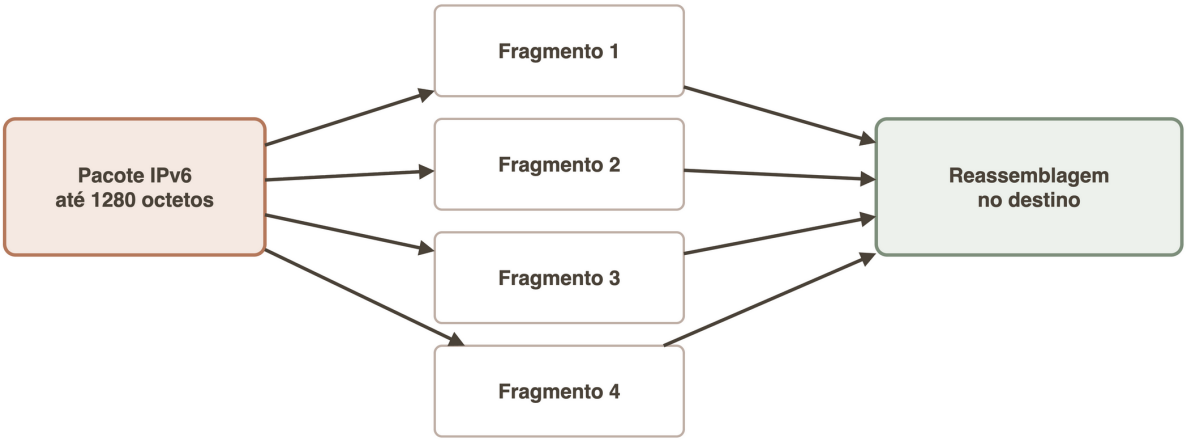
A linha com `bytes` apresenta o cabeçalho comprimido em hexadecimal. Associe cada octeto ao diagrama. Por que razão os endereços globais fazem o cabeçalho passar de 6 para 38 octetos?

Erro possível e resolução

Em Windows, o comando `sed` não existe. Executar apenas o programa e ler as linhas correspondentes.

Passo 4 — Fragmentação e perdas

Objetivo. Estimar quantas tramas são necessárias para uma mensagem e o efeito da perda de tramas.



Perder um fragmento obriga a retransmitir o datagrama completo

Como ler a figura

O pacote IPv6, à esquerda, é dividido em fragmentos, ao centro, que viajam separadamente e são reunidos no destino, à direita. Se um fragmento se perder, o destino não consegue reconstituir o pacote e este tem de ser retransmitido por inteiro.

Fundamento teórico

Cada fragmento acrescenta 4 ou 5 octetos de cabeçalho de fragmentação, e o espaço de dados de cada fragmento é múltiplo de 8 octetos. O programa assume um cálculo simplificado, que ignora detalhes como o desvio do primeiro fragmento. Se a probabilidade de perda de uma trama for p e a mensagem usar n tramas, a probabilidade de entrega é $(1-p)^n$, que decresce rapidamente com n.

A analisar neste passo: a diferença que a compressão faz no número de tramas, e o efeito que isso tem na fiabilidade.

Comando a executar

```
~/tciot-lab09/bin/python sixlowpan_lab.py 20 | sed -n 8,13p
```

Resultado esperado (os valores podem variar)

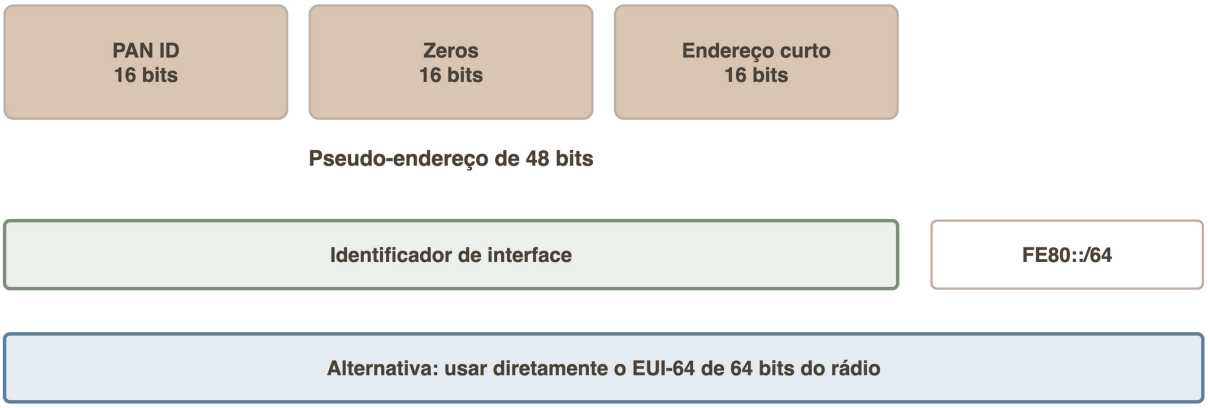
```
payload  20 octets: 1 frame(s) uncompressed, 1 compressed
payload  60 octets: 2 frame(s) uncompressed, 1 compressed
payload 150 octets: 3 frame(s) uncompressed, 2 compressed
payload 400 octets: 5 frame(s) uncompressed, 5 compressed
loss 1% per frame: success with 1 frame 0.99, with 5 frames 0.95
loss 10% per frame: success with 1 frame 0.90, with 5 frames 0.59
```

Analisar

Para que dimensões de mensagem a compressão reduz o número de tramas? Para uma perda de 10% por trama, que redução de fiabilidade resulta de passar de 1 para 5 tramas?

Passo 5 — O identificador de interface

Objetivo. Verificar a construção do identificador de interface a partir do identificador da PAN e do endereço curto.



Como ler a figura

A figura mostra os três blocos do pseudo-endereço de 48 bits: o identificador da PAN, 16 bits nulos e o endereço curto. Por baixo, o identificador de interface e o prefixo de ligação local `FE80::/64`.

Fundamento teórico

Quando o dispositivo só dispõe do endereço curto, o identificador de interface é construído com a PAN, 16 bits nulos e o endereço curto, com `FFFE` inserido a meio e o bit universal/local a zero, que significa âmbito local. O resultado não é globalmente único: só tem significado dentro da PAN.

A analisar neste passo: onde aparece cada parte do endereço curto no identificador final.

Comando a executar

```
~/tciot-lab09/bin/python sixlowpan_lab.py | tail -1
```

Resultado esperado (os valores podem variar)

```
interface id for PAN 0xABCD, short 0x0001: a9cd:00ff:fe00:0001
```

Analisar

Altere, no final do programa, o identificador da PAN e o endereço curto, e preveja o resultado antes de o executar. Que endereço de ligação local obtém?

Passo 6 — Limpeza

Comando a executar

```
rm -r ~/tciot-lab09
```

Questões de consolidação

1. Que fração de uma trama com segurança é carga útil quando não há compressão?
2. Porque é que a compressão só chega a 6 octetos em ligação local?
3. Porque é que a compressão sem estado interessa a dispositivos com pouca memória?
4. Para uma perda de 10% por trama, qual a diferença de fiabilidade entre uma mensagem de 1 trama e uma de 5 tramas?
5. Porque é que, no caso de estudo da aula, não foi usado IP?

Síntese

- O **orçamento** de uma trama 802.15.4 é pequeno: 54 octetos de carga útil sem segurança e 33 com ela.
- A **compressão** sem estado reduz 48 octetos de cabeçalho a 6, em ligação local.
- Com **endereços globais** a compressão é parcial, e o cabeçalho cresce.
- A **fragmentação** permite mensagens maiores, mas a probabilidade de entrega decresce com o número de tramas.